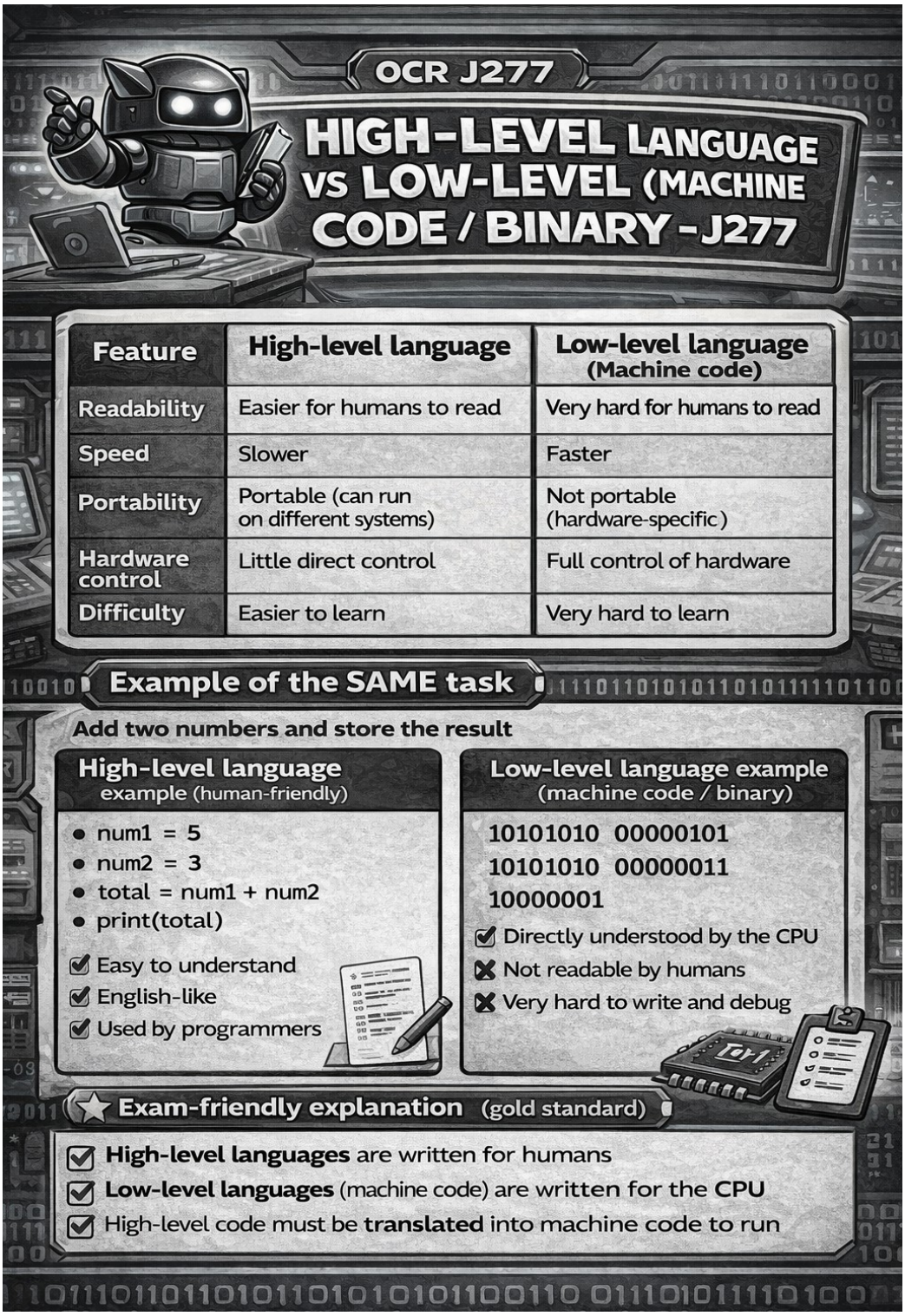
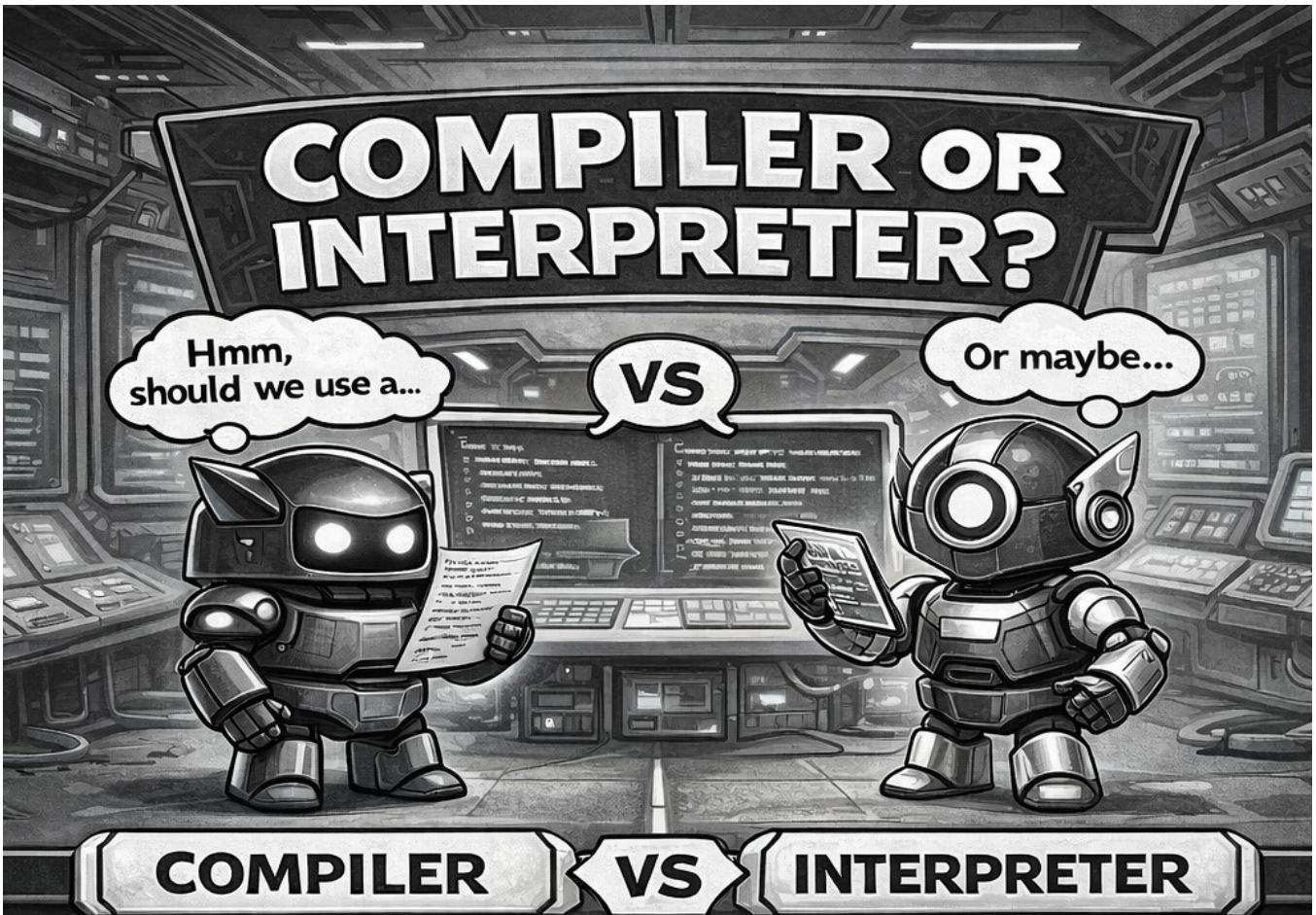
2.5 Programming Languages and Integrated Development Environments

Keyword	Definition
Programming Language	A language which can be understood by computers to produce computer software.
High Level Language	A programming language which is close to English in the commands used making it easier to learn and understand.
Low Level Language	A programming language closer to machine code, using less English like commands making it harder to understand.
Translator	A piece of software which converts programming language into machine code which the computer can understand and execute.
Compiler	A translator which converts code in one go.
Interpreter	A translator which converts code line by line.
Integrated Development Environment	A piece of software used by developers to write computer code. They often contain features such as translators and syntax highlighters to make it easier to write code.
Editor	A piece of software used to write computer code.
Error Diagnostics	Tools to help locate and resolve errors in code.
Runtime Environment	A tool which allows code to run on a system for which it was not designed. For example, running a mobile phone app on a desktop computer.

2.5.1 Languages

2.5.1 Languages	
☐ Characteristics and purpose of different levels of programming language: ○ High-level languages ○ Low-level languages ☐ The purpose of translators ☐ The characteristics of a compiler and an interpreter	**Required** ✓ The differences between high-level and low-level programming languages ✓ The need for translators ✓ The differences, benefits and drawbacks of using a compiler or an interpreter **Not required** × Understanding of assemblers





COMPILER VS INTERPRETER

- **What it does**
- Translates the **whole program** in one go
- Produces **an executable file**
- Reports **all errors** at once
- Program runs fast after compilation

Characteristics

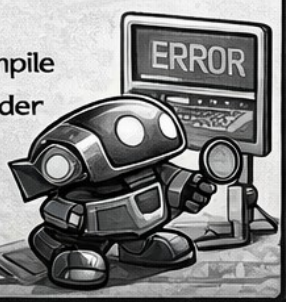
- Translates the whole program in onego
- Produces an executable file
- Reports all errors at once
- Program runs fast after compilation

Benefits

- Faster execution
- Errors shown together
- No translator needed once compiled

Drawbacks

- Takes time to compile
- Errors may be harder to locate
- Must recompile after changes



- **What it does**
- An interpreter **translates** and runs the program **line by line**
- Translates one line at a time.
- No executable file created.

Benefits

- Easier to debug
- Errors shown one at a time
- Good for learning and testing

Drawbacks

- Slower execution
- Translator needed every time
- Errors only shown as they occur

- **Drawbacks**
- Slower execution
- Translator needed every time



2.5.1 Exam Questions on Languages

Question	Tip to answer
Which type of language is easiest for humans to read and write? (1) ☐ Machine code ☐ Assembly ☐ High-level language ☐ Hexadecimal	Humans
High-level language.[1]	
Which translator converts high-level code into machine code before running the program? (1) ☐ Interpreter ☐ Compiler ☐ Assembler ☐ Debugger	"Before running
Compiler.[1]	
State one reason why machine code is difficult for humans to use. (1)	It's 1s and 0s?
It is written in binary (1s and 0s), so it is hard to read and write.[1]	
Explain the difference between a high-level language and a low-level language. (3)	3 points: readability, hardware dependence, translation.
High-level languages use English-like keywords so they are easier for humans to read and write.[1] Low-level languages (like assembly/machine code) are closer to the CPU instructions and are harder for humans.[2] High-level code must be translated into machine code before the CPU can run it.[3]	
Explain why translators are needed. (3)	CPU only understands machine code.
The CPU can only understand machine code (binary).[1] Translators convert high-level code into machine code so the program can run.[2] Without a translator, the CPU would not know what the high-level instructions mean.[3]	
Compare a compiler and an interpreter. (4)	2 points each: how they translate + pros/cons.
A compiler translates the whole program into machine code before it runs.[1] This usually makes the program run faster once compiled.[2] An interpreter translates and runs the code line by line.[3] This makes it easier to spot errors quickly because it stops where the error happens.[4]	
A company is creating software for a portable games console. Recommend whether they should use a compiler or an interpreter, and justify your choice. (6)	Use TIER style: pick one, give benefits + drawback.
A compiler is best for a portable games console because the program can be translated into machine code before it is shipped.[1] This means the game will run faster and more smoothly on the device.[2] It also means the console does not need to translate code while playing, saving processing power and battery.[3] A compiled program can be easier to distribute because users just run the executable.[4] However, compiling takes longer when developing because you must recompile after changes.[5] Overall, the faster performance is the key reason for using a compiler on a console.[6]	

02 KEY VOCABULARY / 2.5.1

Languages and translators

Use the correct technical wording before attempting longer answers.

High-level language

Uses human-readable constructs and abstracts away hardware detail.

Low-level language

Closer to machine instructions and hardware.

Compiler

Typically translates a program as a whole before it runs.

Interpreter

Typically translates and executes code during execution.

Assembler

Translates assembly instructions into machine code.

03 WORKED EXAMPLE / 2.5.1

Languages and translators

Follow a complete example. Notice the method before attempting the separate questions.

Task
Choose a suitable translator for a program.

Think it through — small steps

1. A high-level program is easier to read and maintain than machine instructions.
2. A compiler typically translates the source program before running it.
3. An interpreter typically executes by translating instructions as the program runs.
4. The right choice depends on testing, portability and execution needs.

Why this earns marks
A comparison explains how each translator operates rather than just naming it.

SPEC 2.5.1 | SKILL 27 OF 28

Languages and translators

REMEMBER

High-level languages are more readable; low-level languages are closer to machine instructions. Translators turn source into executable form.

THINK OF IT LIKE...

Write instructions for people, then translate them for the machine.

WORKED EXAMPLE / APPLY

A compiler commonly translates a program ahead of execution; an interpreter executes translated instructions as it runs.

YOU DO - EXAM PRACTICE

State one difference between a compiler and an interpreter. (2)

Write your answer before opening the separate answer section.

04 INDEPENDENT EXAM PRACTICE / 2.5.1

Languages and translators

Do this without viewing the next answer pages. Use a whiteboard first for code/traces.

EXAM-STYLE CHALLENGE · 6 MARKS

Explain two differences between high-level and low-level languages, then explain one difference between a compiler and an interpreter. Use clear technical terminology.

Your response

PREDICT → WRITE → TEST → IMPROVE | Self-mark after finishing the entire topic.

MARK SCHEME | SKILL 27 OF 28

Languages and translators

QUESTION

State one difference between a compiler and an interpreter. (2)

MODEL ANSWER

A compiler typically translates a whole source program before running it (1). An interpreter typically translates and executes instructions during execution (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

05 ANSWERS & MARK SCHEME / 2.5.1

Languages and translators

This answer belongs to the exam-style challenge on the previous practice page.

6-mark challenge · Model answer
Compare languages and translators

Mark 1 High-level languages are generally easier for humans to read and write.

Mark 2 Low-level languages map more closely to hardware instructions.

Mark 3 High-level languages are generally more portable across processor architectures.

Mark 4 Low-level language may require architecture-specific knowledge or rewriting.

Mark 5 A compiler typically translates the whole program before execution.

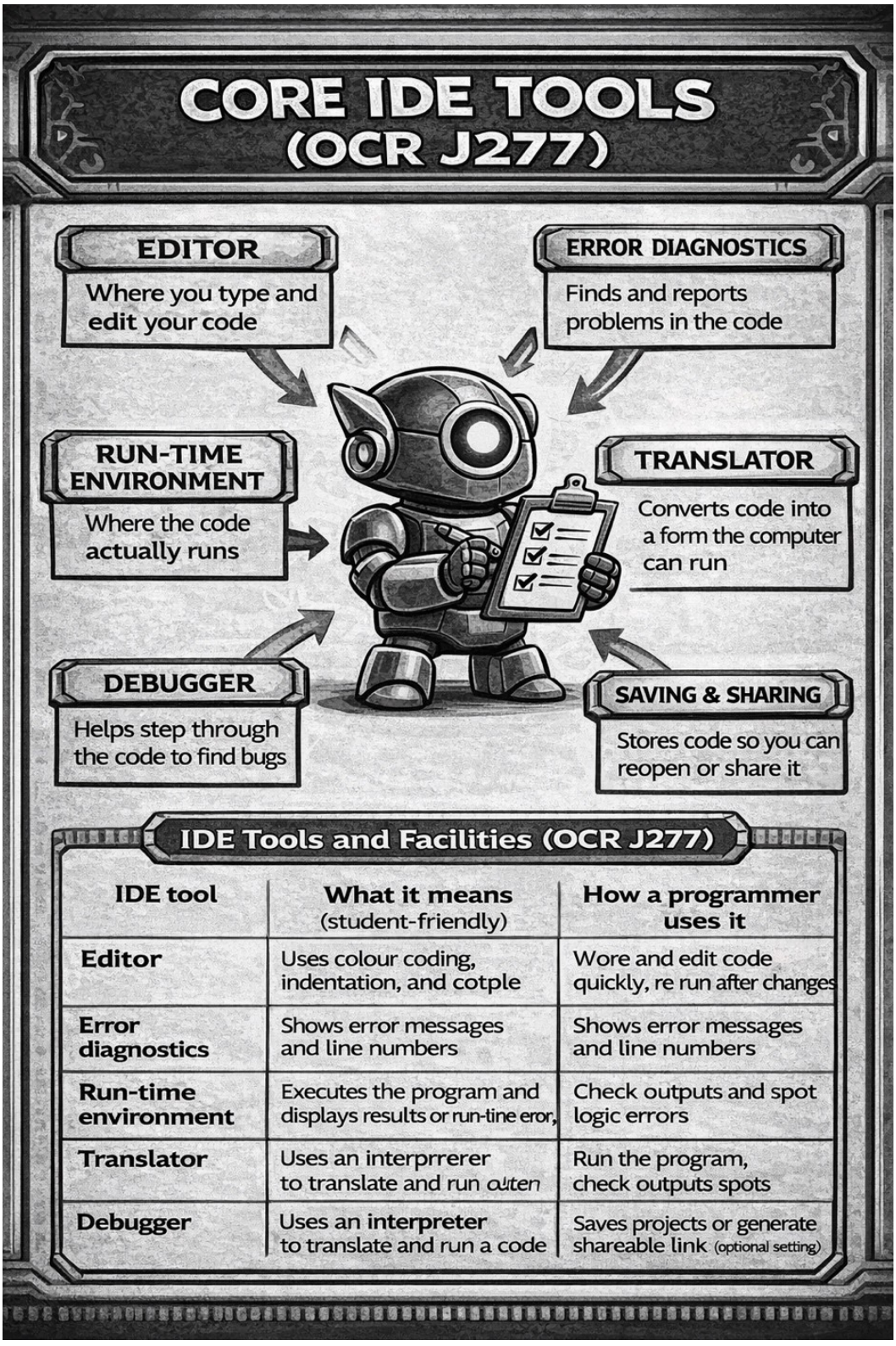
Mark 6 An interpreter typically translates and executes instructions during execution.

Accept equivalent correct methods, code or relevant wording. Award each point once.

2.5.2 The Integrated Development Environment (IDE)

2.5.2 The Integrated Development Environment (IDE)	
☐ Common tools and facilities available in an Integrated Development Environment (IDE): ○ Editors ○ Error diagnostics ○ Run-time environment ○ Translators	**Required** ✓ Knowledge of the tools that an IDE provides ✓ How each of the tools and facilities listed can be used to help a programmer develop a program ✓ Practical experience of using a range of these tools within at least one IDE

IDE tools help programmers **write code faster, find errors more easily,** and **improve program quality.**



IDE tool	What it means	How the IDE provides it	How a programmer uses it	Common exam mistakes
Editor	Where you type and edit your code	Uses colour coding, indentation, and autocomplete	Write and edit code quickly, re-run after changes	Saying the editor ***runs*** code (it doesn't — it only edits)
Error diagnostics	Messages that explain what went wrong and	Shows error messages and line numbers	Find the error, fix it, run again	Confusing error diagnostics with the debugger

Run-time environment	Where the program runs and shows output	Executes the program and displays results or run-time errors	Check outputs and spot logic errors	Forgetting that logic errors appear here, not as syntax errors
Translator	Converts code into a form the computer can execute	Uses an **interpreter** to translate and run code	Test code behaviour exactly as it would execute	Saying the IDE ***is*** the translator rather than ***contains*** one
Debugger	Helps find errors by stepping through code	Runs code line by line and shows variable values	Track how values change to find bugs	Saying it only finds syntax errors (it's mainly for logic errors)
Sharing / saving work	Allows programs to be saved or shared	Saves projects or generates shareable links	Reopen work later or share with others	Forgetting this is a ***useful feature***, not a core programming tool

2.5.2 Questions on the Integrated Development Environment (IDE)

Question	Tip to answer (student)
State one type of error that can happen in a program. (1)	Any one: syntax / logic / runtime.
Syntax error.[1]	
Which error happens when the program breaks while it is running (e.g. divide by zero)? (1) ☐ Syntax error ☐ Runtime error ☐ Logic error ☐ Semantic error	"While running" = runtime.
Runtime error.[1]	
Explain what a syntax error is. (2)	2 points: breaks language rules + stops running/compiling.
A syntax error is when code breaks the rules of the language (wrong keyword/symbol/indentation).[1] The program will not compile or run until it is fixed.[2]	
Explain what a logic error is. (2)	2 points: runs but wrong output.
A logic error is when the program runs but the algorithm is wrong.[1] This means it produces the wrong output/results.[2]	
A student writes: `if score >= 5 AND <= 10 then` Explain why this causes an error and correct it. (3)	Must compare the variable twice.
It is wrong because <= 10 is not a full comparison (it is missing the variable).[1] The computer cannot tell what should be compared to 10.[2] Correct example: if score >= 5 AND score <= 10 then[3]	
Describe two IDE tools that help find and fix errors. (4)	2 tools + what each does.
Debugger (breakpoints/stepping) lets you run code line-by-line to see where it goes wrong.[1] Variable watch shows the values of variables while the program runs so you can spot incorrect values.[2] Syntax highlighting / error messages point out where syntax mistakes are in the code.[3] Autocomplete suggests correct keywords and reduces typing mistakes.[4]	
A program is meant to add up 1 to 10, but it outputs 45. Identify the likely type of error and explain how you would locate it using an IDE. (6)	Name error type + stepping + watch variables + check loop bounds.
This is most likely a **logic error** because the program runs but gives the wrong total.[1] I would use the debugger to step through the loop one line at a time.[2] I would add a breakpoint at the start of the loop so it stops each iteration.[3] I would use a variable watch to check the running total and the loop counter as they change.[4] I would check the loop bounds (e.g. starting at 0 or stopping too early) and the update line (e.g. adding the wrong value).[5] Once I find the line where the total becomes wrong, I would correct the calculation or the loop range and run again to test.[6]	

Resources

Common Exam Questions – worked through

02 KEY VOCABULARY / 2.5.2

Integrated development environments

Use the correct technical wording before attempting longer answers.

Code editor

Lets a programmer enter and modify source code.

Syntax highlighting

Uses colours or styles to distinguish code elements.

Debugger

Helps locate faults during execution.

Breakpoint

Pauses execution at a chosen line.

Variable watch

Displays a variable value during debugging.

Auto-completion

Suggests valid commands, names or syntax while typing.

03 WORKED EXAMPLE / 2.5.2

Integrated development environments

Follow a complete example. Notice the method before attempting the separate questions.

Task
Investigate a wrong total with an IDE.

Think it through — small steps

1. Set a breakpoint inside the loop that updates total.
2. Run the program until the breakpoint pauses execution.
3. Step through each statement in the debugger.
4. Watch the value of total and compare it with the expected trace.
5. Correct the faulty condition or calculation and retest.

Why this earns marks
A named IDE tool is linked to a specific debugging action and evidence.

SPEC 2.5.2 | SKILL 28 OF 28

Integrated development environments

REMEMBER

An IDE combines coding tools such as an editor, debugger and syntax highlighting.

THINK OF IT LIKE...

A workshop containing your coding tools in one place.

WORKED EXAMPLE / APPLY

Set a breakpoint to pause execution and inspect variable values.

YOU DO - EXAM PRACTICE

Name two IDE features that help a programmer find errors. (2)

Write your answer before opening the separate answer section.

04 INDEPENDENT EXAM PRACTICE / 2.5.2

Integrated development environments

Do this without viewing the next answer pages. Use a whiteboard first for code/traces.

EXAM-STYLE CHALLENGE · 6 MARKS

A student finds that a program runs but gives the wrong total. Describe three IDE features and explain how each could help the student investigate or correct this problem.

Your response

PREDICT → WRITE → TEST → IMPROVE | Self-mark after finishing the entire topic.

MARK SCHEME | SKILL 28 OF 28

Integrated development environments

QUESTION

Name two IDE features that help a programmer find errors. (2)

MODEL ANSWER

Debugger/breakpoints (1) and syntax highlighting or error diagnostics (1). Other relevant features may be credited.

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

05 ANSWERS & MARK SCHEME / 2.5.2

Integrated development environments

This answer belongs to the exam-style challenge on the previous practice page.

6-mark challenge · Model answer
Choose IDE features

Mark 1 Feature: breakpoint.

Mark 2 Use: pauses the program at a chosen line.

Mark 3 Feature: debugger / single stepping.

Mark 4 Use: executes one statement at a time to locate the incorrect step.

Mark 5 Feature: variable watch / inspector.

Mark 6 Use: shows changing values so a wrong total can be spotted.

Accept equivalent correct methods, code or relevant wording. Award each point once.